T5_OPENMP:

Q1.
Explanation: Code 1: Without `critical`

- The `#pragma omp parallel for` directive distributes loop iterations among multiple threads.
- Each thread computes `f(i)` and adds the result to `v[indices[i]]`.
- If two or more iterations access the same array index, a race condition may occur because multiple threads can update the same element simultaneously.
- Therefore, this code is correct only if concurrent iterations do not update the same array element, or if appropriate synchronization is provided elsewhere.

Code 2: With `critical`

- The `critical` directive ensures that only one thread at a time executes the enclosed block.
- This prevents race conditions when multiple iterations update the same array element.
- However, it reduces parallelism because only one thread can execute the critical section at a time.
- Therefore, this code is correct for valid array indices, thinking that no other unsynchronized conflicting accesses occur.

Q2.

2(a).The first loop executes $f(i)$, while the second loop executes $f(n - i)$. Since the execution time of $f(i)$ is i milliseconds, iterations with larger values of i take longer to execute.

With schedule(static), the iterations are divided into two large chunks, one assigned to each thread. Consequently, the threads may have different workloads.

At the end of each omp for loop, OpenMP has an implicit barrier. A thread that finishes its assigned iterations early must wait until the other thread finishes.

For an example illustration, assume $n = 9$, so the loop iterations run from 1 to 8.

First loop: f(i)

- Thread 0 executes $f(1), f(2), f(3), f(4)$.
- T_0=1+2+3+4=10ms
- Thread 1 executes $f(5), f(6), f(7), f(8)$.
  T_1=5+6+7+8=26ms

Thread 0 finishes first and waits for $26 - 10 = 16$ ms. The first loop therefore takes 26 ms of elapsed time.

Second loop: f(n-i)

The workload is reversed:

- Thread 0 executes $f(8), f(7), f(6), f(5)$, taking 26 ms.
- Thread 1 executes $f(4), f(3), f(2), f(1)$, taking 10 ms.

Thread 1 waits for $26 - 10 = 16$ ms. The second loop also takes 26 ms of elapsed time.

Thread No	1st loop work	1st loop wait	2nd loop work	2nd loop wait
Thread 0	10	16	26	0
Thread 1	26	0	10	16

2(b).

Using schedule(static,1)

The clause schedule(static,1) assigns one iteration at a time to threads in round-robin order.

For the first loop with $n = 9$:

- Thread 0 executes $f(1), f(3), f(5), f(7)$, taking $1 + 3 + 5 + 7 = 16$ ms.
- Thread 1 executes $f(2), f(4), f(6), f(8)$, taking $2 + 4 + 6 + 8 = 20$ ms.

Thread 0 waits for $20 - 16 = 4$ ms.

The second loop reverses the workload, so the thread workloads become 20 ms and 16 ms. The elapsed time for each loop is 20 ms.

Therefore, `schedule(static,1)` improves load balancing and reduces the waiting time

2. Using schedule(dynamic,1)

With schedule(dynamic,1), each thread receives one iteration at a time and requests another iteration when it finishes its current one.

This scheduling method can balance uneven workloads more effectively. However, it introduces additional scheduling overhead.

The nowait clause removes the implicit barrier at the end of a loop.

For example:

```
#pragma omp for schedule(static) nowait

for (i = 1; i < n; i++)

    f(i);
```

Without nowait, both threads must finish the first loop before proceeding to the second loop. With nowait, a thread can start the next loop immediately after finishing its current iterations.

The intermediate waiting time is eliminated.

Conclusion:

The default static scheduling can cause waiting when threads receive workloads of different sizes.

Using schedule(static,1) improves workload distribution, while schedule(dynamic,1) may improve load balancing at the cost of additional overhead. The nowait clause removes the intermediate implicit barrier, allowing threads to proceed without waiting for each other at that point.

Q3.

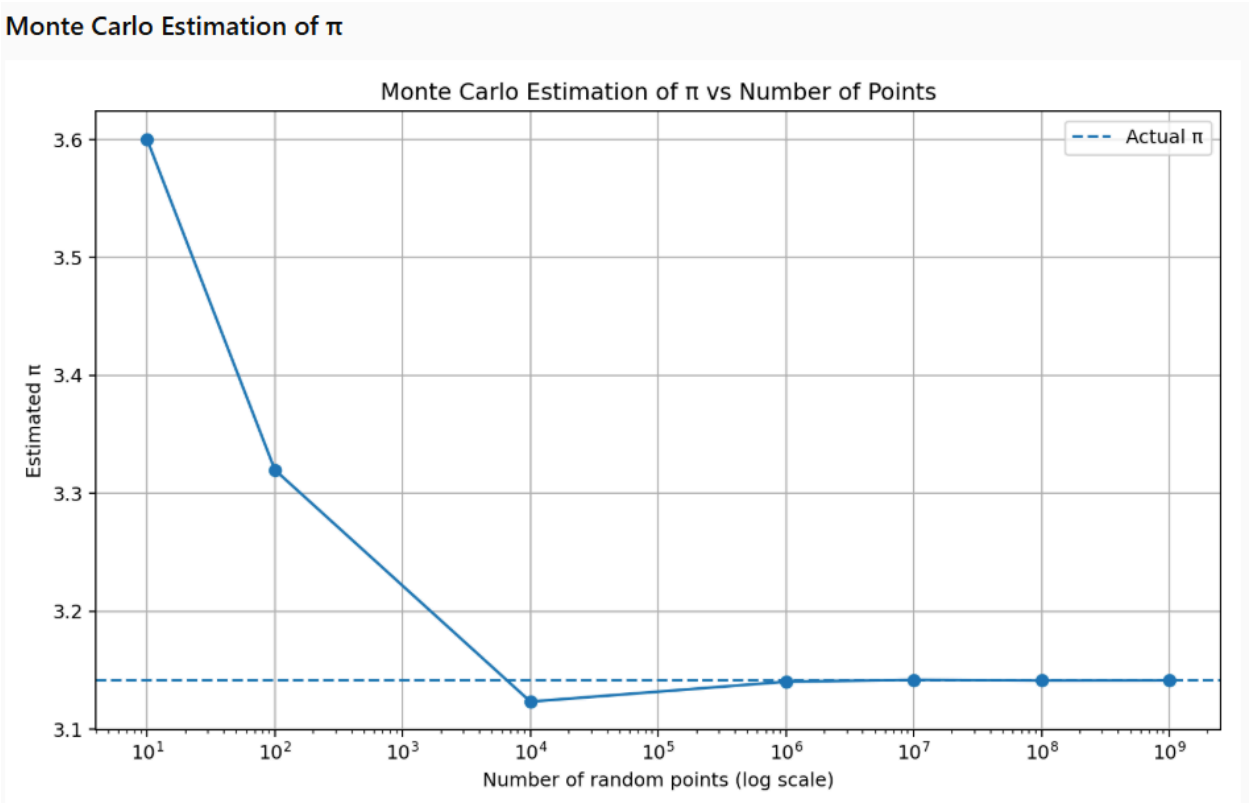
Let us take a circle inside a square.

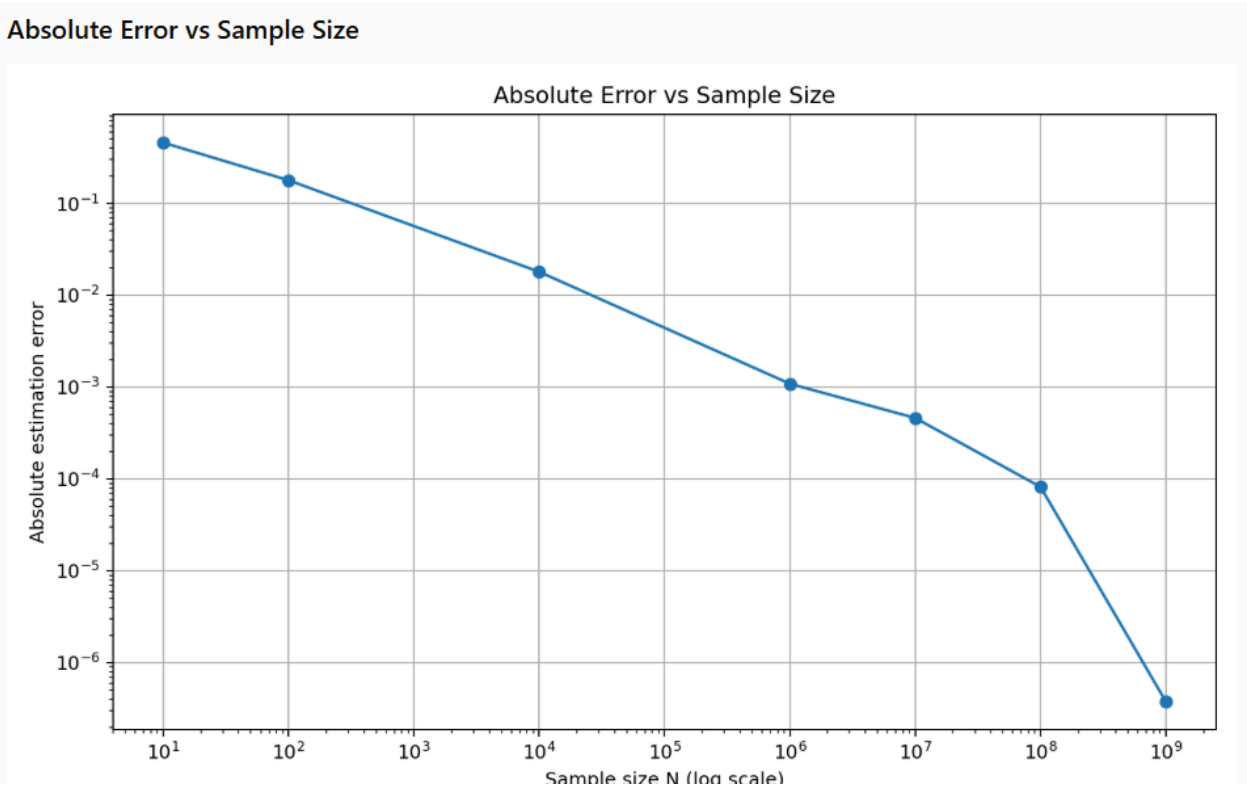
- The circle has radius R.
- The square has side length 2R.
- The area of the circle is pi R^2.
- The area of the square is 4R^2.

The probability that a random point inside the square also lies inside the circle is the ratio of their areas:

$$P=\pi R^2/4 R^2 = \pi/4$$

So; pi=4*points inside circle/total random points





The absolute error generally decreases as the sample size increases .

Conclusion:

Code is in Git Repo with attached link in submission.

The Monte Carlo method was implemented using OpenMP to estimate the value of π. Random points were generated within a square, and the number of points lying inside the inscribed circle was counted using parallel reduction. The estimated value was calculated using the formula:

π = 4 × (Number of points inside the circle / Total number of points)

The experiment was performed for sample sizes ranging from 10 to 10^9 points. The results show that the estimated value approaches the actual value of π as the number of random points increases. The absolute estimation error generally decreases with increasing sample size, demonstrating the effectiveness of the Monte Carlo method for numerical estimation.

Q4.

Code is in git repo attached in link submission.

Thread safety and efficient parallelism: Thread safety is achieved by storing the left and right subtree sums in separate result variables and using the OpenMP taskwait directive before combining the results. This prevents the program from reading incomplete results. Efficient parallelism is achieved by using OpenMP tasks to process independent subtrees concurrently and limiting unnecessary task creation to reduce overhead.

Q5.

Code is in git repo attached in link submission.

1. Breadth-First Search (BFS)

Breadth-First Search is a graph traversal algorithm that visits vertices level by level, starting from a given source vertex. It visits all neighboring vertices before moving to the next level. In the OpenMP implementation, vertices in the current level are processed in parallel using #pragma omp parallel for. A shared visited array prevents vertices from being visited more than once.

2. Depth-First Search (DFS)

Depth-First Search is a graph traversal algorithm that explores a branch as far as possible before returning to explore other branches. In the OpenMP implementation, #pragma omp task is used to explore neighboring vertices concurrently. The traversal continues until all reachable vertices have been visited.

3. Thread Safety

Thread safety ensures that multiple threads can execute simultaneously without causing incorrect results. In these implementations, OpenMP critical sections protect shared data, such as the visited array and the BFS frontier, from conflicting concurrent updates. The taskwait directive in DFS ensures that child tasks complete before the current task finishes.

4. OpenMP Parallelism

OpenMP is an API that supports shared-memory parallel programming in C. It provides compiler directives such as parallel for, task, critical, and taskwait to distribute work among threads and

coordinate their execution. Parallelism can reduce execution time when the workload is sufficiently large, although synchronization and task-creation overhead may affect performance.

Q6. Testing default scheduling taking it as dynamic:

Code is in git repo attached in link submission.

1. A parallel loop containing 20 iterations was executed using four OpenMP threads.
2. No scheduling clause was specified in the omp for directive.
3. The function omp_get_thread_num() was used to identify the thread executing each iteration.
4. The iteration-to-thread assignments were printed and analyzed.

Observation:

The output showed the following distribution:

- Thread 0 executed iterations 0–4.
- Thread 1 executed iterations 5–9.
- Thread 2 executed iterations 10–14.
- Thread 3 executed iterations 15–19.

Each thread executed five consecutive iterations.

Result: The observed iteration distribution is consistent with static scheduling, in which iterations are divided into chunks and assigned to threads.

Conclusion: The experiment demonstrated how OpenMP distributes loop iterations among threads when no scheduling clause is specified. The observed output followed a contiguous-block pattern consistent with static scheduling. However, the output alone does not conclusively establish the runtime's default scheduling policy; this should be confirmed using the documentation for the OpenMP implementation.

Additional Question:

Code is in git repo attached in link submission.

The plate is initialized to 50°C. The left, top, and right boundaries are maintained at 100°C, and the bottom boundary is maintained at 0°C. The temperature of each interior cell is iteratively calculated as the average of its four neighboring cells.

The OpenMP parallel for directive distributes the interior-cell calculations among multiple threads. Two temperature grids are used to ensure that all calculations in an iteration read the previous temperature values. The reduction(max:maxChange) clause calculates the maximum temperature change across all threads.

The simulation stops when the maximum temperature change becomes smaller than the specified tolerance or when the maximum iteration count is reached.

Independent interior-cell calculations are executed concurrently, while the boundary conditions remain fixed. The simulation approaches a steady-state temperature distribution.